



High-Performance Computer Architectures Practical Course

Advanced C++ and Project Tools

Prof. Dr. Ivan Kisel

Robin Lakos

Akhil Mithran

Oddharak Tyagi

May 7, 2025

We would like to express our gratitude to Francesco Giacomini of INFN, Bologna, for generously permitting us to use his C++ course slides as inspiration for our introductory C++ lectures in this course.

- Object oriented C++ (class, struct, template, overloading)
- Header- and source-files
- Linking
- Build tools and meta-build tools
- Optimization levels and compiler flags

Classes in C++

A class in C++ is a blueprint for creating objects, defining the data attributes and behaviors (methods) that the objects of the class will have.

It encapsulates data for object-oriented programming, allowing for data abstraction, inheritance, and polymorphism, thus managing the complexity of larger software systems.

```
class MyClass
{
private:
    TYPE myVariable_;

public:
    MyClass(); // constructor
    ~MyClass(); // destructor

    TYPE my_method(...);
}

TYPE MyClass::my_method(...) { ... }
```

In C++, you can write the variable and method definitions inside the class, which is often done for so called header-only libraries. However, for object-oriented C++, especially in larger projects, this is not suggested.

– This topic is covered later in this lecture.

A struct in C++ is a user-defined data type that allows the grouping of variables of different types under a single name, facilitating data management and organization.

Structs are primarily used for data storage and are similar to classes, but they default to public access for their members and base classes.

```
struct MyStruct
{
    TYPE x_;
    TYPE y_;
    TYPE z_;

    TYPE my_method(...);
}

TYPE MyStruct::my_method(...) { ... }
```

Difference to classes: The main difference between a struct and a class in C++ is the default access level; members and base classes in structs are public by default, whereas in classes, they are private.

Templates in C++

In the previous lecture, we already discussed templates briefly. Templates in C++ can be quite useful in situations like type-independent container classes or functions that work the same way on different types.

On the other hand, they can also be used for other "flexible" parameters, similar to functions that expect a value.

```
// primary template for the Fibonacci calculation
template <int N>
struct Fibonacci {
    static const int value = Fibonacci<N - 1>::value + Fibonacci<N - 2>::value;
};

// specializations for base cases
template <>
struct Fibonacci<0> {
    static const int value = 0;
};

template <>
struct Fibonacci<1> {
    static const int value = 1;
};

int main() {
    std::cout << "Fibonacci(10): " << Fibonacci<10>::value << std::endl;
    return 0;
}
```

Class- and Function-Templates (1)

Example:

```
// MyContainer.h

#ifndef MY_CONTAINER_H
#define MY_CONTAINER_H

template <typename T>
class MyContainer {
private:
    T* data;
    size_t size;
public:
    MyContainer(size_t n);
    void push_back(const T& value);
    T& operator[](size_t index);
    size_t getSize() const;
};

#endif // MY_CONTAINER_H
```

Example:

```
// Template function example:
template <typename T>
auto max_value(const T& container) {
    auto max = container[0];
    for (size_t i = 1; i < container.size(); ++i) {
        if (container[i] > max) {
            max = container[i];
        }
    }
    return max;
}
```

Function and Operator Overloading in C++

Function and operator overloading in C++ allows multiple functions or operators to have the same name but differ in parameters or behavior.

```
class Vector2D {
public:
    double x_, y_;

    Vector2D(double x, double y) : x_(x), y_(y) {}

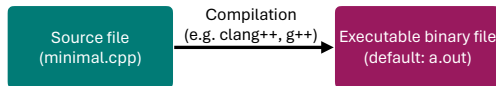
    // Method to add two vectors
    Vector2D add(const Vector2D& v) {
        return Vector2D(x_ + v.x_, y_ + v.y_);
    }

    // Operator "+" overloading
    Vector2D operator+(const Vector2D& v) {
        return add(v);
    }
};

// Vector2D v1(1.0, 2.0);
// Vector2D v2(3.0, 4.0);
// Vector2D v3 = v1 + v2; // Uses overloaded "+", instead of "v1.add(v2);"
```


The C++ code compilation for single source-files in the last lecture was simplified:

Translation into executable:



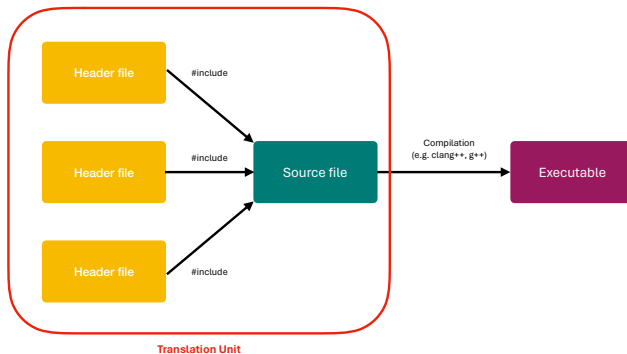
For the minimal C++ application, this simplification was true. However, even for the simple "Hello world!"-example, we had to include other libraries to manage the input-output-stream.

– What happens to files included using the `#include`-statement?

Compilation Model in C++ (2)

What actually happens is that so-called *translation units* are created as a first step of the compilation process.

Building translation units:



The creation of translation units is accomplished by running the preprocessor on a source file.

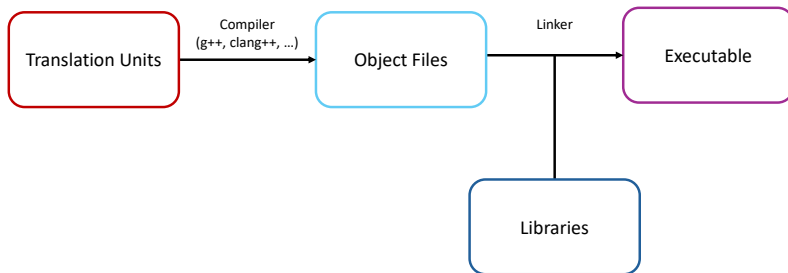
Please note: One can stop the compilation after preprocessing by using compiler flags (e.g. `-E` in `g++`).

Compilation Model in C++ (3)

The `#include`-statement is (and was in the previous exercise) used to include header files. These files usually contain the declarations of variables and functions.

- A larger project is composed of many files and, therefore, many translation units
- Functionality from one translation unit should be usable by other translation units

In C++, the linker allows to link to other libraries to make their functionality available in the project.



Please note: One can stop the compilation after compilation by using compiler flags (e.g. `-c` in `g++`, and to save temporary files use `-save-temps`).

Please note: C++ 20 introduced *modules* which are not further discussed in this lecture.

In C++, software components are typically split into up to three different files:

- A header file that contains free function declarations, class definitions with member function (method) declarations and template definitions.
- A source file containing the definitions of free functions and member functions (methods)
- A file containing the unit tests of the component

Functions defined in a header file are required to be `inline`. Member functions (methods) defined inside the class and function templates are implicitly `inline`.

Please note: The `inline` keyword is not (any more) an optimization hint to the compiler.

Header- and Source-Files (2)

Example:

```
1 // ExampleClass.h
2
3 #ifndef EXAMPLE_CLASS_H
4 #define EXAMPLE_CLASS_H
5
6 class ExampleClass
7 {
8 private:
9     int data;
10
11 public:
12     ExampleClass();
13
14     void setData(int value);
15     int getData() const;
16 };
17
18 #endif // EXAMPLE_CLASS_H
```

```
1 // ExampleClass.cpp
2
3 #include "ExampleClass.h"
4
5 ExampleClass::ExampleClass()
6 {
7     data = 0;
8 }
9
10 void ExampleClass::setData(int value)
11 {
12     data = value;
13 }
14
15 int ExampleClass::getData() const
16 {
17     return data;
18 }
```

Linking is the process of combining multiple object files into a single executable or library. It resolves references between different parts of the program. There are two main types of linking: static linking and dynamic linking.

Static Linking

- The linker combines object files and libraries at compile time.
- Produces a standalone executable file.
- All code and libraries are included in the executable.

Dynamic Linking

- Libraries are linked at runtime.
- Shared libraries (DLLs in Windows, shared objects in Unix-like systems).
- Reduces executable size.
- Allows for easier updates of libraries.

Example:

- To link with an external library, use '-L' to specify the library directory and '-l' to specify the library name.
- Example: `g++ -o myprogram myprogram.cpp -L/path/to/library -lmylibrary`
- This links the library named `mylibrary` located in `/path/to/library` during compilation.

Build tools automate the process of compiling source code into executable programs. They manage dependencies and invoke the compiler with the correct options (Example: Make, Ninja)

Make

- Make is a classic build tool that reads a Makefile to determine how to compile and link a program.
- It supports parallel execution of tasks and is widely used in Unix-like environments.

Meta-build tools generate build files for specific build systems. They abstract away platform-specific details and provide a uniform interface for building projects (Example: CMake).

CMake

- CMake is a meta-build tool that generates Makefiles, Visual Studio projects, or other build system files based on a high-level description.
- It simplifies the build process by providing a platform-independent way to define build configurations.

```
# CMakeLists.txt

cmake_minimum_required(VERSION 3.27)
project(GameOfLife)

set(CMAKE_CXX_STANDARD 17)

set(CMAKE_CXX_FLAGS_RELEASE "${CMAKE_CXX_FLAGS_RELEASE} -O3")

include_directories(include)

add_executable(GameOfLife
    GameOfLife.cpp
    src/World.cpp
    src/CLInterface.cpp
)

# command to build using CMake
mkdir build      # create build directory
cd build        # move into build directory
cmake -DCMAKE_BUILD_TYPE=Release ..    # meta-build with cmake file in previous directory
make            # build
```


Compiler flags are important to control the application compilation process (e.g. -g for debugging), but they also have a severe impact to the runtime performance of the program.

Example:

The compiler flag "-O3" enables the highest optimization level. The compiler tries to optimize the code as good as possible when creating the executable. However, it can also include mechanisms "hidden" to the programmer to optimize the code.

It enables aggressive optimization techniques, including loop unrolling, function inlining, and instruction-level parallelism like SIMD (Single Instruction, Multiple Data) instructions and optimizations by-hand of the developer might not result in expected performance boosts.

Although compilers do a good job, their optimization is not always the best. Later on, we will disable some compiler optimizations to see the real performance boosts of own implementations.

Exercise - Cellular Automaton: Conway's Game of Life

Conway's Game of Life is a classic example of a cellular automaton (CA). It consists of a grid of cells, each of which can be in one of two states: alive or dead. The state of each cell evolves over discrete time steps based on simple rules.

Despite its simplicity, Conway's Game of Life exhibits complex and interesting behavior, including stable patterns, oscillators, and gliders.

Why a Cellular Automaton?

Prof. Dr. I. Kisel and his group developed extremely efficient and highly parallel tracking algorithms that are based on the idea of the Cellular Automaton:

- These algorithms help reconstruct particle trajectories from the detector data.
- By modeling the detector as a grid of cells, similar to Conway's Game of Life, these algorithms can efficiently identify particle tracks and vertices.
- CA-based tracking algorithms offer robustness and scalability, making them suitable for the high-density and high-volume data environment of heavy-ion experiments.
- These algorithms play a crucial role in extracting physics signals from the complex detector data in heavy-ion collision experiments.